Lab 8 Edge Detection

Abbas Abdelaziz Al-Tunayan
Python and OpenCV

Apply Sobel, Prewitt, and Canny to a new image, then investigate how Canny thresholds change the detected edges. The selected image is **cow.png**, a 774 x 429 pixel cow photograph supplied for this task.

Task 1 Edge Detection Results



Processing. Sobel and Prewitt use grayscale followed by a 5 x 5 Gaussian blur. Their gradient magnitudes are normalized to 0-255 for display. Canny uses grayscale with thresholds 100 and 200, without an additional explicit blur, following the reference example.

Observation. Sobel and Prewitt show broad, varying-strength responses on the cow outline, coat patches, and grass. Canny gives thin binary edges and suppresses weaker changes. The smooth sky has fewer strong responses than the textured grass.

Task 1 Code

Load the new image and compute the three requested maps. Normalizing before conversion to uint8 avoids integer wraparound.

```
from pathlib import Path
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Put cow.png beside this notebook before choosing Run All.
image_path = Path('cow.png')
if not image_path.is_file():
    raise FileNotFoundError('Upload cow.png beside this notebook.')

img = cv2.imread(str(image_path), cv2.IMREAD_COLOR)
if img is None:
    raise ValueError('OpenCV could not read cow.png.')
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
output_dir = Path('results')
output_dir.mkdir(exist_ok=True)

def to_uint8(response):
    # Scale before casting to prevent wraparound above 255.
    return cv2.normalize(
        response, None, 0, 255, cv2.NORM_MINMAX
    ).astype(np.uint8)

print(f'Selected image: cow.png ({img.shape[1]} x {img.shape[0]})')

# Sobel: first derivatives in x and y.
sobel_x = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize=3)
sobel = to_uint8(cv2.magnitude(sobel_x, sobel_y))

# Prewitt: manually defined 3 x 3 derivative kernels.
kernel_x = np.array([[-1, 0, 1], [-1, 0, 1], [-1, 0, 1]],
                    dtype=np.float64)
kernel_y = np.array([[-1, -1, -1], [0, 0, 0], [1, 1, 1]],
                    dtype=np.float64)
prewitt_x = cv2.filter2D(blurred, cv2.CV_64F, kernel_x)
prewitt_y = cv2.filter2D(blurred, cv2.CV_64F, kernel_y)
prewitt = to_uint8(cv2.magnitude(prewitt_x, prewitt_y))

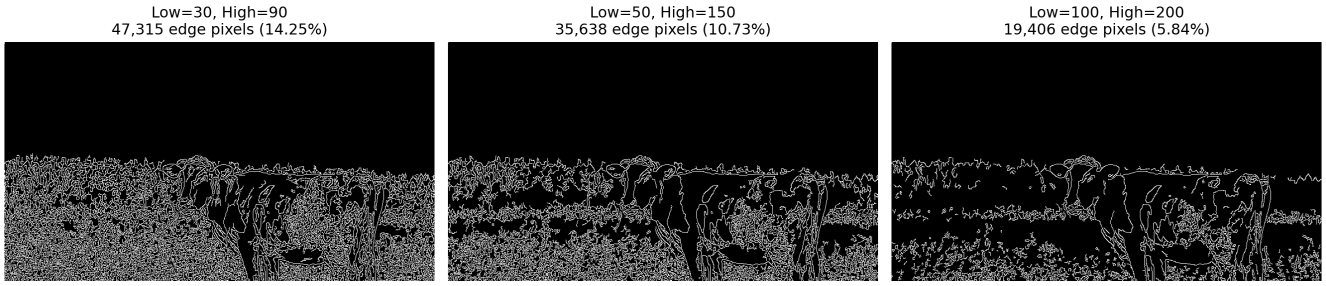
# Canny: same grayscale input as the threshold experiment.
canny = cv2.Canny(gray, 100, 200)
```

Display the original and the three edge maps

```
fig, axes = plt.subplots(2, 2, figsize=(12, 7.8))
panels = [rgb, sobel, prewitt, canny]
titles = ['Original image: cow.png', 'Sobel', 'Prewitt',
          'Canny: low=100, high=200']
for ax, panel, title in zip(axes.flat, panels, titles):
    ax.imshow(panel, cmap='gray', vmin=0, vmax=255)
    ax.set_title(title)
    ax.axis('off')
fig.tight_layout()
fig.savefig(output_dir / 'task1_methods.png', dpi=160)
plt.show()
plt.close(fig)
```

Task 2 Effect of Canny Thresholds

Only the thresholds change in this experiment. All three runs use the same grayscale image and default Canny settings.



Low	High	Edge pixels	Image percentage
30	90	47,315	14.25%
50	150	35,638	10.73%
100	200	19,406	5.84%

30 and 90. This is the most sensitive setting. Many weak grass details and small variations remain visible, producing 47,315 edge pixels. It preserves detail but may also retain unwanted texture or noise.

50 and 150. Some weak responses disappear while the cow and many grass details remain visible. The intermediate result contains 35,638 edge pixels.

100 and 200. Fewer weak details survive and strong boundaries are emphasized. The map contains 19,406 edge pixels. Low-contrast boundaries can become incomplete or disappear.

The edge-pixel count decreased by **59.0%** between 30/90 and 100/200. Counts measure sensitivity, not detection accuracy; no ground-truth edge labels were supplied.

Task 2 Code and Explanation

```
threshold_pairs = [(30, 90), (50, 150), (100, 200)]
canny_maps = {}
edge_counts = []
fig, axes = plt.subplots(1, 3, figsize=(15, 4.3))

for ax, (low, high) in zip(axes, threshold_pairs):
    # Only the thresholds change; gray and defaults stay fixed.
    edges = cv2.Canny(gray, low, high)
    canny_maps[(low, high)] = edges
    count = int(np.count_nonzero(edges))
    percent = 100 * count / edges.size
    edge_counts.append((low, high, count, percent))
    ax.imshow(edges, cmap='gray', vmin=0, vmax=255)
    ax.set_title(f'Low={low}, High={high}\n'
                 f'{count:,} edge pixels ({percent:.2f}%)')
    ax.axis('off')
    cv2.imwrite(str(output_dir / f'canny_{low}_{high}.png'), edges)

fig.tight_layout()
fig.savefig(output_dir / 'task2_canny_thresholds.png', dpi=180)
plt.show()
plt.close(fig)

print('Low | High | Edge pixels | Percentage of image')
for low, high, count, percent in edge_counts:
    print(f'{low:3d} | {high:4d} | {count:11,d} | {percent:.2f}%')
```

How the Thresholds Affect the Result

The high threshold identifies strong edge responses. Responses between the low and high thresholds are weak candidates and are retained only when connected through weak candidates to a strong edge. Responses below the low threshold are discarded.

Raising the high threshold makes fewer responses strong edge seeds. Raising the low threshold removes more weak candidates. Lower thresholds preserve more small details but can increase visual clutter. Higher thresholds suppress weak detail but may miss useful edges.

Conclusion

All requested methods were applied to the new cow image. For this image, thresholds 50 and 150 provide an intermediate visual result between the detailed 30/90 map and the sparser 100/200 map. The notebook includes executed outputs for both tasks.